# LAB REPORT: COMPLEX SQL & MINI-PROJECT SYNOPSIS

CSBB 204: Database Management Systems | Week 6 (07/09/2026 – 11/09/2026)

**Name:** Aryan Nautiyal **Roll Number:** 251210015

## Aim

To execute advanced relational SQL operations (pattern matching, arithmetic operations, logical filtering, and joins) and formulate a Mini-Project Database Synopsis for a College T&P portal.

## Theory

| Operator | Description |
|---|---|
| `LIKE '%'` | Matches any string of zero or more characters (wildcard). |
| `LIKE '_'` | Matches exactly one single character at the specified index position. |
| `BETWEEN a AND b` | Specifies an inclusive boundary range (`val ⩾ a AND val ⩽ b`). |
| `+, -, *, /, %` | Standard arithmetic operators executed directly in projections or update clauses. |
| `AND, OR, NOT` | Boolean logical operators used to combine multiple selection conditions in `WHERE`. |
| `CONCAT(s1, s2)` | String function to combine multiple column values into a single formatted string. |
| `INNER JOIN` | Relational join returning rows where matching keys exist across joined tables. |

### (a) Employees in department 4 whose first name starts with 'A'

```
MariaDB [company]> SELECT fname, lname, dno FROM employee
    -> WHERE dno = 4 AND fname LIKE 'A%';
+--------+--------+-----+
| fname  | lname  | dno |
+--------+--------+-----+
| Alicia | Zelaya |   4 |
+--------+--------+-----+
1 row in set (0.000 sec)
```

### (b) Employees whose first name ends with 'n'

```
MariaDB [company]> SELECT fname, lname FROM employee
    -> WHERE fname LIKE '%n';
+----------+-------+
| fname    | lname |
+----------+-------+
| John     | Smith |
| Franklin | Wong  |
+----------+-------+
2 rows in set (0.000 sec)
```

### (c) Employees whose first name contains 'm' (case-insensitive)

```
MariaDB [company]> SELECT fname, lname FROM employee
    -> WHERE fname LIKE '%m%';
+--------+---------+
| fname  | lname   |
+--------+---------+
| Ramesh | Narayan |
| Ahmad  | Jabbar  |
| James  | Borg    |
```

```
+--------+---------+
3 rows in set (0.000 sec)
```

**(d) Employees whose first name has exactly five characters**

```
MariaDB [company]> SELECT fname, lname FROM employee
    -> WHERE fname LIKE '_____';
+-------+---------+
| fname | lname   |
+-------+---------+
| Joyce | English |
| Ahmad | Jabbar  |
| James | Borg    |
+-------+---------+
3 rows in set (0.000 sec)
```

**(e) Employees whose last name contains 'a' at the second position**

```
MariaDB [company]> SELECT fname, lname FROM employee
    -> WHERE lname LIKE '_a%';
+----------+---------+
| fname    | lname   |
+----------+---------+
| Jennifer | Wallace |
| Ramesh   | Narayan |
| Ahmad    | Jabbar  |
+----------+---------+
3 rows in set (0.000 sec)
```

**(f) Calculate the annual salary of each employee**

```
MariaDB [company]> SELECT fname, lname, salary, (salary * 12) AS annual_salary
    -> FROM employee;
+----------+-----------+----------+---------------+
| fname    | lname     | salary   | annual_salary |
+----------+-----------+----------+---------------+
| John     | Smith     | 34500.00 |     414000.00 |
| Franklin | Wong      | 46000.00 |     552000.00 |
| Robert   | Scott     | 66700.00 |     800400.00 |
| Alicia   | Zelaya    | 28750.00 |     345000.00 |
| Jennifer | Wallace   | 49450.00 |     593400.00 |
| Ramesh   | Narayan   | 43700.00 |     524400.00 |
| Joyce    | English   | 28750.00 |     345000.00 |
| Ahmad    | Jabbar    | 28750.00 |     345000.00 |
| James    | Borg      | 63250.00 |     759000.00 |
| Anirudh  | Periasamy |     NULL |          NULL |
+----------+-----------+----------+---------------+
10 rows in set (0.000 sec)
```

**(g) Employees whose salary is between 30000 and 50000**

```
MariaDB [company]> SELECT fname, lname, salary FROM employee
    -> WHERE salary BETWEEN 30000 AND 50000;
+----------+---------+----------+
| fname    | lname   | salary   |
+----------+---------+----------+
| John     | Smith   | 34500.00 |
| Franklin | Wong    | 46000.00 |
| Jennifer | Wallace | 49450.00 |
| Ramesh   | Narayan | 43700.00 |
```

```
+----------+---------+----------+
4 rows in set (0.000 sec)
```

**(h) Increase the salary of all employees by 20%**

```
MariaDB [company]> UPDATE employee SET salary = salary * 1.20;
Query OK, 9 rows affected (0.003 sec)
Rows matched: 10  Changed: 9  Warnings: 0
```

**(i) Deduct 10000 from the salary of all employees**

```
MariaDB [company]> UPDATE employee SET salary = salary - 10000;
Query OK, 9 rows affected (0.002 sec)
Rows matched: 10  Changed: 9  Warnings: 0
```

**(j) Calculate the salary of all employees after a 10% deduction**

```
MariaDB [company]> SELECT fname, lname, salary, ROUND(salary * 0.90, 2) AS
salary_after_10_percent_cut
    -> FROM employee;
+----------+-----------+----------+-----------------------------+
| fname    | lname     | salary   | salary_after_10_percent_cut |
+----------+-----------+----------+-----------------------------+
| John     | Smith     | 31400.00 |                    28260.00 |
| Franklin | Wong      | 45200.00 |                    40680.00 |
| Robert   | Scott     | 70040.00 |                    63036.00 |
| Alicia   | Zelaya    | 24500.00 |                    22050.00 |
| Jennifer | Wallace   | 49340.00 |                    44406.00 |
| Ramesh   | Narayan   | 42440.00 |                    38196.00 |
| Joyce    | English   | 24500.00 |                    22050.00 |
| Ahmad    | Jabbar    | 24500.00 |                    22050.00 |
| James    | Borg      | 65900.00 |                    59310.00 |
| Anirudh  | Periasamy |     NULL |                        NULL |
+----------+-----------+----------+-----------------------------+
10 rows in set (0.000 sec)
```

**(k) Divide the salary of all employees by 3**

```
MariaDB [company]> SELECT fname, lname, salary, ROUND(salary / 3, 2) AS salary_div_by_3
    -> FROM employee;
+----------+-----------+----------+-----------------+
| fname    | lname     | salary   | salary_div_by_3 |
+----------+-----------+----------+-----------------+
| John     | Smith     | 31400.00 |        10466.67 |
| Franklin | Wong      | 45200.00 |        15066.67 |
| Robert   | Scott     | 70040.00 |        23346.67 |
| Alicia   | Zelaya    | 24500.00 |         8166.67 |
| Jennifer | Wallace   | 49340.00 |        16446.67 |
| Ramesh   | Narayan   | 42440.00 |        14146.67 |
| Joyce    | English   | 24500.00 |         8166.67 |
| Ahmad    | Jabbar    | 24500.00 |         8166.67 |
| James    | Borg      | 65900.00 |        21966.67 |
| Anirudh  | Periasamy |     NULL |            NULL |
+----------+-----------+----------+-----------------+
10 rows in set (0.000 sec)
```

**(l) Remainder of the salary when divided by 5000**

```
MariaDB [company]> SELECT fname, lname, salary, MOD(salary, 5000) AS remainder_mod_5000
    -> FROM employee;
```

```
+----------+-----------+----------+--------------------+
| fname    | lname     | salary   | remainder_mod_5000 |
+----------+-----------+----------+--------------------+
| John     | Smith     | 31400.00 |            1400.00 |
| Franklin | Wong      | 45200.00 |             200.00 |
| Robert   | Scott     | 70040.00 |              40.00 |
| Alicia   | Zelaya    | 24500.00 |            4500.00 |
| Jennifer | Wallace   | 49340.00 |            4340.00 |
| Ramesh   | Narayan   | 42440.00 |            2440.00 |
| Joyce    | English   | 24500.00 |            4500.00 |
| Ahmad    | Jabbar    | 24500.00 |            4500.00 |
| James    | Borg      | 65900.00 |             900.00 |
| Anirudh  | Periasamy |     NULL |               NULL |
+----------+-----------+----------+--------------------+
10 rows in set (0.000 sec)
```

**(m) First name of employees whose salary is less than 30000**

```
MariaDB [company]> SELECT fname, salary FROM employee
    -> WHERE salary < 30000;
+--------+----------+
| fname  | salary   |
+--------+----------+
| Alicia | 24500.00 |
| Joyce  | 24500.00 |
| Ahmad  | 24500.00 |
+--------+----------+
3 rows in set (0.000 sec)
```

**(n) Last name of employees whose salary is greater than 20000**

```
MariaDB [company]> SELECT lname, salary FROM employee
    -> WHERE salary > 20000;
+---------+----------+
| lname   | salary   |
+---------+----------+
| Smith   | 31400.00 |
| Wong    | 45200.00 |
| Scott   | 70040.00 |
| Zelaya  | 24500.00 |
| Wallace | 49340.00 |
| Narayan | 42440.00 |
| English | 24500.00 |
| Jabbar  | 24500.00 |
| Borg    | 65900.00 |
+---------+----------+
9 rows in set (0.000 sec)
```

**(o) Complete name of employees with salary > 10000 and department number = 5**

```
MariaDB [company]> SELECT CONCAT(fname, ' ', minit, '. ', lname) AS complete_name, salary,
dno
    -> FROM employee
    -> WHERE salary > 10000 AND dno = 5;
+------------------+----------+-----+
| complete_name    | salary   | dno |
+------------------+----------+-----+
| John B. Smith    | 31400.00 |   5 |
| Franklin T. Wong | 45200.00 |   5 |
| Ramesh K. Narayan | 42440.00 |   5 |
```

```
+------------------+----------+-----+
3 rows in set (0.000 sec)
```

**(p) First name and address of employees in department 1 or 4**

```
MariaDB [company]> SELECT fname, address, dno FROM employee
    -> WHERE dno IN (1, 4);
+----------+---------------------------------+-----+
| fname    | address                         | dno |
+----------+---------------------------------+-----+
| Robert   | 2365 Newcastle Rd, Bellaire, TX |   1 |
| Alicia   | 3321 Castle, Spring, TX         |   4 |
| Jennifer | 291 Berry, Bellaire, TX         |   4 |
| Joyce    | 5631 Rice, Houston, TX          |   4 |
| Ahmad    | 980 Dallas, Houston, TX         |   1 |
| James    | 450 Stone, Houston, TX          |   1 |
| Anirudh  | NULL                            |   1 |
+----------+---------------------------------+-----+
7 rows in set (0.000 sec)
```

**(q) Salary of all employees who are not in department 4**

```
MariaDB [company]> SELECT fname, lname, salary, dno FROM employee
    -> WHERE dno <> 4;
+----------+-----------+----------+-----+
| fname    | lname     | salary   | dno |
+----------+-----------+----------+-----+
| John     | Smith     | 31400.00 |   5 |
| Franklin | Wong      | 45200.00 |   5 |
| Robert   | Scott     | 70040.00 |   1 |
| Ramesh   | Narayan   | 42440.00 |   5 |
| Ahmad    | Jabbar    | 24500.00 |   1 |
| James    | Borg      | 65900.00 |   1 |
| Anirudh  | Periasamy |     NULL |   1 |
+----------+-----------+----------+-----+
7 rows in set (0.000 sec)
```

**(r) Employees whose salary lies in inclusive range of 5000 and 28000**

```
MariaDB [company]> SELECT fname, lname, salary FROM employee
    -> WHERE salary BETWEEN 5000 AND 28000;
+--------+---------+----------+
| fname  | lname   | salary   |
+--------+---------+----------+
| Alicia | Zelaya  | 24500.00 |
| Joyce  | English | 24500.00 |
| Ahmad  | Jabbar  | 24500.00 |
+--------+---------+----------+
3 rows in set (0.000 sec)
```

**(s) Project numbers from `project` table between 2 and 25**

```
MariaDB [company]> SELECT pnumber, pname FROM project
    -> WHERE pnumber BETWEEN 2 AND 25;
+---------+-----------------+
| pnumber | pname           |
+---------+-----------------+
|       2 | ProductY        |
|       3 | ProductZ        |
|       4 | ProductA        |
|      10 | Computerization |
```

```
+---------+-----------------+
4 rows in set (0.000 sec)
```

**(t) Modify `hours` in `works_on` for `essn = '999887777'` and `pno` between 10 and 50**

```
MariaDB [company]> UPDATE works_on
    -> SET hours = 25.0
    -> WHERE essn = '999887777' AND pno BETWEEN 10 AND 50;
Query OK, 2 rows affected (0.001 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

**(u) Project name and number where manager start date is on or before 1995-01-01**

```
MariaDB [company]> SELECT p.pname, p.pnumber, d.dnumber, d.mgr_start_date
    -> FROM project p
    -> INNER JOIN department d ON p.dnum = d.dnumber
    -> WHERE d.mgr_start_date ⩽ '1995-01-01';
+-----------------+---------+---------+----------------+
| pname           | pnumber | dnumber | mgr_start_date |
+-----------------+---------+---------+----------------+
| ProductA        |       4 |       4 | 1995-01-01     |
| Computerization |      10 |       4 | 1995-01-01     |
| Allbenefits     |      30 |       4 | 1995-01-01     |
+-----------------+---------+---------+----------------+
3 rows in set (0.001 sec)
```

## DBMS Mini-Project Synopsis

**Aim**

The T&P office requires an automated online platform to eliminate manual document verification, streamline corporate recruitment drives, and maintain student eligibility tracking. The DBMS must support role-based operations across three user categories: Students, Recruiters, and T&P Officers (TPO).

**Modules**

| Name | Functionality |
|---|---|
| **Authentication & RBAC** | Handles encrypted credential validation, password hashing, and Role-Based Access Control (TPO, Student, Recruiter). |
| **Student Profile** | Enables students to record academic details (CGPA, active backlogs), skills, and manage dynamic links to PDF resumes/CVs. |
| **Job Posting** | Allows verified corporate recruiters to publish job drives with detailed criteria: minimum CGPA, eligible branches, stipend, and deadlines. |
| **Eligibility & Search** | Automates applicant eligibility verification via query predicates and allows recruiters to filter applicant pools based on CGPA and skills. |
| **Application & Tracking** | Maintains recruitment progress (Applied, Shortlisted, Technical Interview, Selected) with candidate tracking. |
| **TPO Reports** | Generates statistical insights (branch-wise placement percentage, salary package distribution, company participation). |

**Conceptual Schema**

- **STUDENT** (student_id, roll_no, fname, lname, email, phone, branch, cgpa, backlogs, passing_year, resume_url, placement_status)
- **STUDENT_SKILL** (student_id, skill_name)
- **COMPANY** (company_id, company_name, industry_domain, contact_email, website, tier_level, is_verified)
- **JOB_POSTING** (job_id, company_id(f), job_title, ctc_lpa, min_cgpa, max_backlogs, eligible_branch, application_deadline, drive_date)
- **APPLICATION** (app_id, student_id(f), job_id(f), applied_timestamp, current_status, remarks)

- **TPO_OFFICER** (officer_id, staff_name, email, role, phone)
- **SELECTION_RESULT** (offer_id, app_id(f), offer_letter_ref, acceptance_status, joining_date)

Note: Underlined attributes represent Primary Keys; (f) denote Foreign Keys.

**Entity Relationships**

- **COMPANY to JOB_POSTING (1 : N):** One company hosts multiple recruitment drives over time.
- **STUDENT to JOB_POSTING (M : N):** Many students apply to multiple drives; modeled through the **APPLICATION** associative entity.
- **STUDENT to STUDENT_SKILL (1 : N):** Multi-valued skill attribute normalized into a composite-key entity.
- **JOB_POSTING to APPLICATION (1 : N):** Each drive receives multiple student candidate submissions.
- **APPLICATION to SELECTION_RESULT (1 : 1):** Each approved final application translates to an offer letter.

**External Schema**

- **Student View:** Access is restricted to personal profile, own application tracking, and viewing job postings where the student meets minimum eligibility.
- **Recruiter View:** Can view candidate qualifications, resumes, and update recruitment rounds. Sensitive student attributes (e.g., student passwords, internal disciplinary remarks) are completely hidden.
- **Administrative View:** Unrestricted full-schema visibility with execution rights to verify recruiters, override student statuses, execute DDL schema adjustments, and generate institutional audits.

**DBMS Features Implemented**

**1. Security Facility & Access Control:**

- RBAC: Using MySQL/MariaDB `GRANT` and `REVOKE` privileges:

```
GRANT SELECT, INSERT ON company_db.APPLICATION TO 'student_role'@'localhost';
GRANT SELECT ON company_db.vw_recruiter_talent_search TO 'recruiter_role'@'%';
```

- Restricts direct access to underlying raw tables, enforcing all reads through security views.

**2. Concurrency Control:**

- Employs InnoDB's Multi-Version Concurrency Control (MVCC) and strict two-phase row-level locking.
- Prevents write-write conflicts when thousands of students submit applications concurrently using atomic transaction blocks:

```
START TRANSACTION;
  INSERT INTO APPLICATION (student_id, job_id, applied_timestamp) VALUES (101, 12, NOW());
COMMIT;
```

**3. View Mechanism:**

- Logical data abstraction that protects student privacy:

```
CREATE VIEW vw_eligible_candidates AS
SELECT s.roll_no, CONCAT(s.fname, ' ', s.lname) AS student_name, s.branch, s.cgpa,
s.resume_url
FROM STUDENT s
WHERE s.placement_status = 'Unplaced';
```

**4. Query Language (SQL):**

- Uses complex DQL with subqueries and multi-table joins to filter candidates based on academic eligibility criteria dynamically.